

Übung 1

Vorbereitung	1
Aufgabe 0: Fakultät	3
Aufgabe 1: Doppelfakultät	3
Aufgabe 2: Primzahlen-Tester	3
Aufgabe 3: Sieb des Eratosthenes	4
Aufgabe 4: Statistische Analyse	5
Aufgabe 5: Bildverarbeitung	5
Aufgabe 6: Theoriefragen (Selbststudium)	8

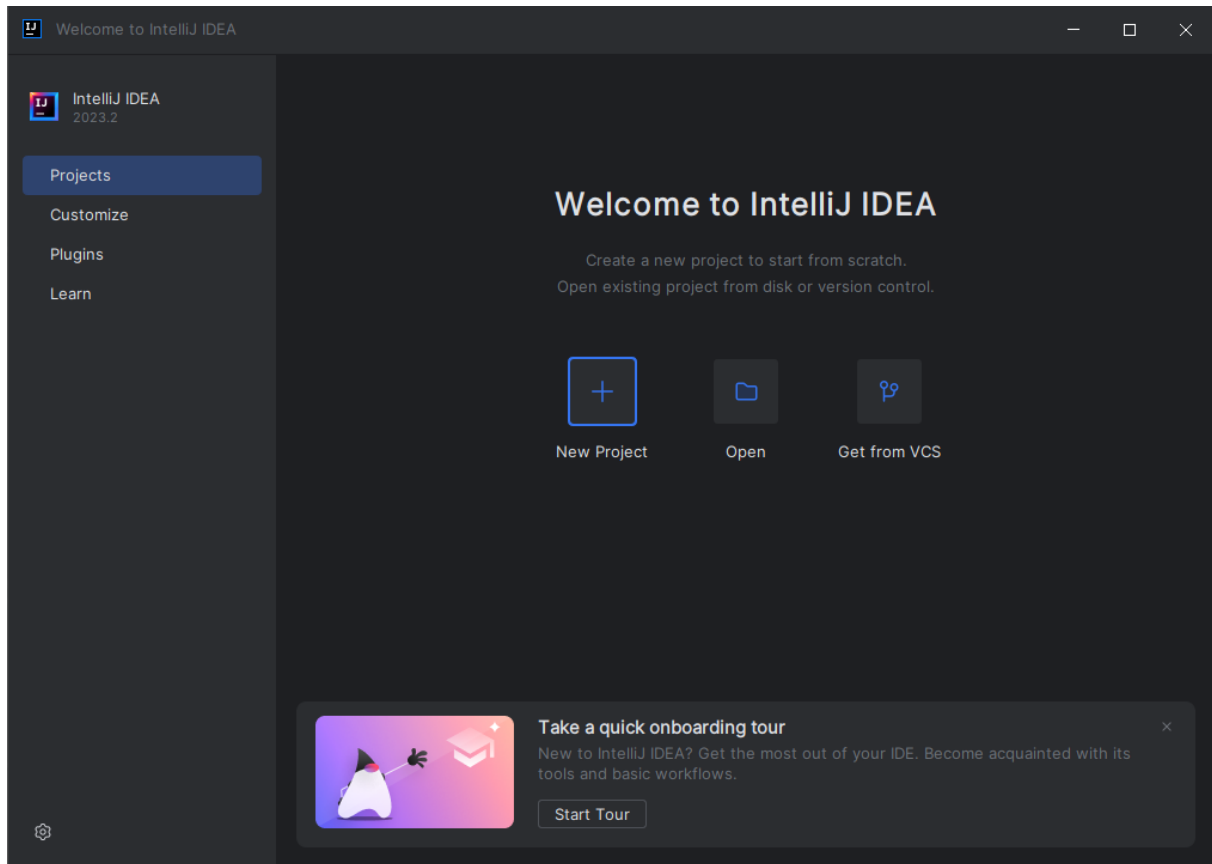
Lernziele

- Einfache imperative Programme zur Lösung von Problemen (Algorithmen) entwerfen.
- Variablen, Datentypen und Kontrollstrukturen in Java anwenden.
- Bedienung der Entwicklungsumgebung kennenlernen.

Vorbereitung

Wir empfehlen Ihnen, IntelliJ IDEA (Community Edition) als Java-IDE zu verwenden. Alternativ, falls Sie bereits mit Eclipse vertraut sind, können Sie auch diese einsetzen. Von anderen IDEs (NetBeans) raten wir eher ab.

- Installieren Sie ein [Java Development Kit, Version 25 LTS](#).
- Laden Sie die aktuelle Version von [IntelliJ IDEA](#) herunter und installieren Sie diese.
 - Alternative Eclipse: Laden Sie den [Eclipse](#) herunter und installieren Sie die «Eclipse IDE for Java Developers». Wählen Sie im zweiten Schritt «Java 25 LTS», das sie vorhin installiert haben.



Auf Moodle steht Ihnen für die Übungen eine Ausgangsbasis als ZIP-Datei zur Verfügung. Entpacken und kopieren Sie zuerst die Ausgangsbasis an einen Ort Ihrer Wahl und importieren Sie diese danach in Ihre IDE.

- IntelliJ:
 1. Open → Wählen Sie den kopierten Ordner als Verzeichnis aus
 2. «Trust Project» klicken um unseren Vorlagen zu vertrauen.
 3. IntelliJ öffnet das Projekt. Unten rechts im Fenster erscheint eine Meldung «Maven build scripts found». Klicken Sie auf dieser Meldung den «Load»-Button um die Maven-Projekte zu laden.
 4. Im Hauptmenü (Hamburger-Menü oben links) finden sie ein Menü-Eintrag «Project Structure». Öffnen Sie dieses und wählen Sie unter «SDK» das installierte JDK 25 LTS aus.
- Eclipse: File → Import → Maven → Existing Maven Projects und wählen Sie als Root Directory den kopierten Ordner mit der Vorlage aus.

Danach werden alle Vorlagen importiert und stehen Ihnen in der IDE zur Verfügung.

Aufgabe 0: Fakultät

In der Vorlesung haben wir die Fakultät berechnet. Sie finden den Code der Vorlesung im Aufgabe 0 Projekt der Übungen

Der Code benutzt eine **for**-Schleife, schreiben Sie ihn um zu einer **while**-Schleife und beantworten Sie danach die folgenden Fragen:

- Welche Variante des Programms finden Sie lesbarer?
- Sehen Sie Vor- oder Nachteile bei der Implementierung mit einer **while**-Schleife?

Aufgabe 1: Doppelfakultät

Nun betrachten wir eine spezielle Variante der Fakultät, die sogenannte Doppelfakultät.

Die Doppelfakultät von n ist definiert als:

$$f(n) = \begin{cases} 1 * 3 * 5 * \dots * (n-2) * n & \text{für ungerade } n \geq 1 \\ 2 * 4 * 6 * \dots * (n-2) * n & \text{für gerade } n \geq 2 \end{cases}$$

Entwickeln Sie ein Programm, das die Doppelfakultät einer Zahl n berechnet und ausgibt. Testen Sie das Programm. Zum Beispiel sollte die Eingabe $n = 10$ das Ergebnis 3840 liefern. Ab wann gibt es einen arithmetischen Überlauf?

Hinweis: Mit geschickter Wahl des Algorithmus braucht es keine Fallunterscheidung.

Aufgabe 2: Primzahlen-Tester

Es soll ein Primzahlen-Test in Java implementiert werden. Dieser soll für eine gegebene Zahl n feststellen, ob die Zahl prim ist. Per Definition ist eine natürliche Zahl $n > 1$ genau dann prim, wenn nur die Zahl 1 oder die Zahl n selbst ein (restloser) Teiler von n ist.

Hinweis: Mit dem Modulo-Operator `%` können Sie den Rest einer Ganzzahl-Division bestimmen. `x % y` ergibt den Rest der Ganzzahl-Division x/y .

Beispiel: $13/2 = 6$, Rest 1. Per Modulo bestimmt man den Rest: $13\%2 = 1$. Generell gilt: Bleibt bei `%2` ein Restwert von 1 übrig, so war der Dividend eine ungerade Zahl.

Aufgabe 3: Sieb des Eratosthenes

Das Sieb des Eratosthenes ist eine altbekannte Methode (sogar noch viel früher erfunden als vom Namensgeber Eratosthenes, 300 v. Chr.). Das Verfahren erlaubt es, eine endliche Folge von Primzahlen ab 2 zu berechnen:

- Man schreibe die ersten N Zahlen in einer Tabelle (Array) auf, z.B. für $N = 20$:

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----

- Die erste Zahl, 2, ist eine Primzahl. Danach streicht man alle Vielfachen dieser Zahl.

2	3		5		7		9		11		13		15		17		19	
---	---	--	---	--	---	--	---	--	----	--	----	--	----	--	----	--	----	--

- Die nächste nicht-gestrichene Zahl, jetzt 3, ist die nächste Primzahl. Danach streicht man wiederum alle Vielfachen dieser Zahl. (Es ist kein Problem, falls eine Zahl schon vorher gestrichen wurde.)

2	3		5		7				11		13				17		19	
---	---	--	---	--	---	--	--	--	----	--	----	--	--	--	----	--	----	--

- Dieses Vorgehen wird so lange wiederholt, bis man die letzte nicht-gestrichene Zahl der Tabelle erreicht hat. Alle nicht-gestrichenen Zahlen sind dann Primzahlen.

Implementieren Sie dieses Verfahren und geben Sie die Primzahlen bis 100 aus (**Achtung:** nicht die ersten 100 Primzahlen).

Tipp: Falls Sie in Versuchung kommen, den Primzahlen-Test aus der vorherigen Aufgabe zu verwenden, sind Sie auf dem falschen Weg. Auch den Modulo-Operator benötigen Sie nicht.

Aufgabe 4: Statistische Analyse

Aus einer Serie a von N Gleitkommazahlen ('double') sollen folgende statistische Werte errechnet werden:

Wert	Berechnung
Minimum / Maximum	
Arithmetischer Mittelwert	$\frac{a_0 + a_1 + \dots + a_{N-1}}{N}$
Varianz	$\frac{1}{N}((a_0 - m)^2 + (a_1 - m)^2 + \dots + (a_{N-1} - m)^2)$, wobei m der arithmetische Mittelwert ist.
Standardabweichung	$\sqrt{var}$, wobei var die Varianz ist.

Entwickeln Sie ein passendes Programm basierend auf der Vorlage.

Hinweis: Benutzen Sie `Math.sqrt()` zur Berechnung der Quadratwurzel.

Optionale Bonusaufgabe: Für die obigen Berechnungen werden oftmals alle Werte betrachtet. Welche Funktionalitäten / Berechnungen könnte man geeignet zusammenfassen, um eine Optimierung zu erzielen? Wann ist dieser Schritt sinnvoll? Wann nicht?

Aufgabe 5: Bildverarbeitung

Ein digitales Bild lässt sich als zweidimensionales Array von Pixelwerten repräsentieren. Jeder Pixelwert (`int`) definiert die Farbe des Pixels.

Nun können folgende Bildverfahren angewendet werden:

- **Invertierung:** Jeder Pixel soll negiert werden, also wird beispielsweise aus dem Wert 11 der Wert -11 und andersherum.
- **Rotation:** Bild um 90 Grad nach rechts rotieren.
- **Spiegelung:** Bild horizontal, also an einer gedachten vertikalen Linie horizontal in der Mitte, spiegeln.

Die Vorlage enthält bereits ein vorbereitetes Java-Projekt eines Bildverarbeitungsprogramms. Sie brauchen lediglich in `ImageProcessingLater` die Verfahren zu implementieren. `ImageViewer` benötigen Sie zur Ausführung, können den Inhalt der Datei aber ansonsten ignorieren.

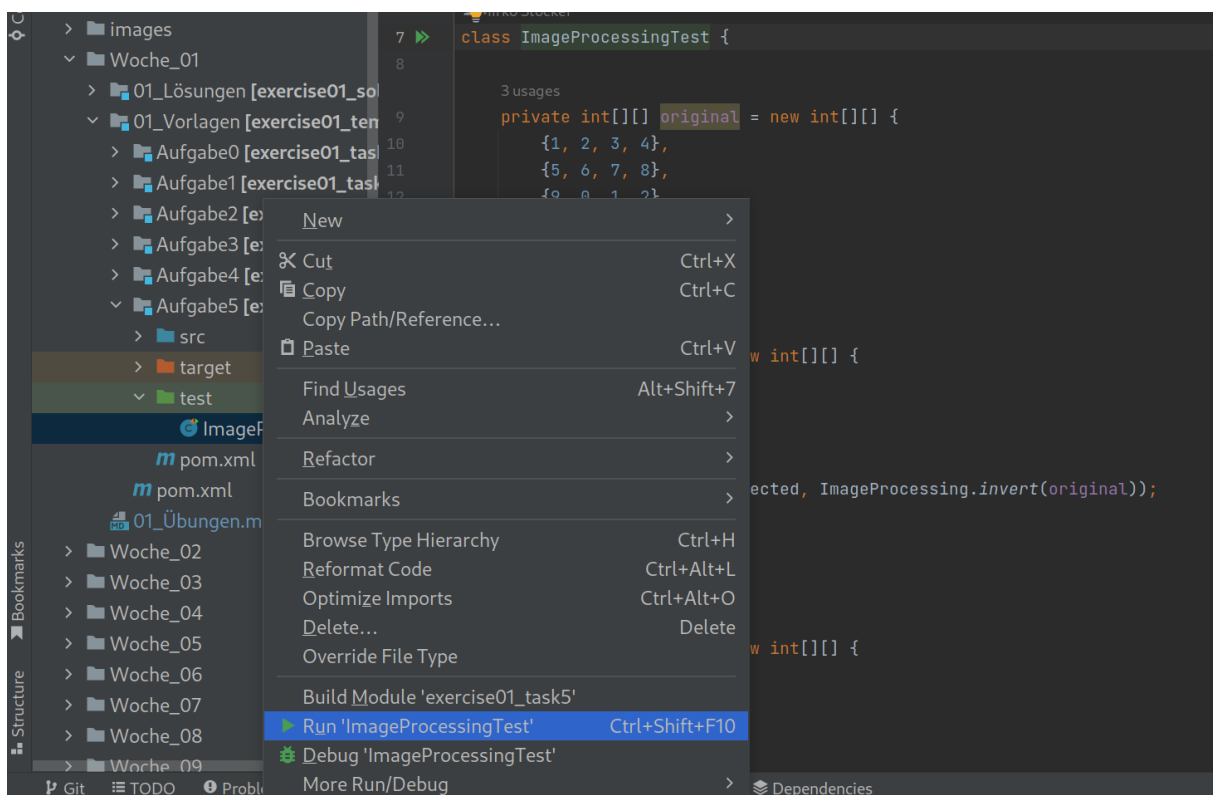
Die Bilder sind wie folgt orientiert: Das äussere Array repräsentiert das ganze Bild. Seine Elemente sind wiederum Arrays, die jeweils eine Zeile des Bilds repräsentieren. Deren Elemente sind `int`-Werte. Jeder `int` repräsentiert einen einzelnen Pixel.

Somit gibt der erste Index des äusseren Arrays die vertikale Position (y-Achse), der zweite Index die horizontale Position (x-Achse) eines Pixels an.

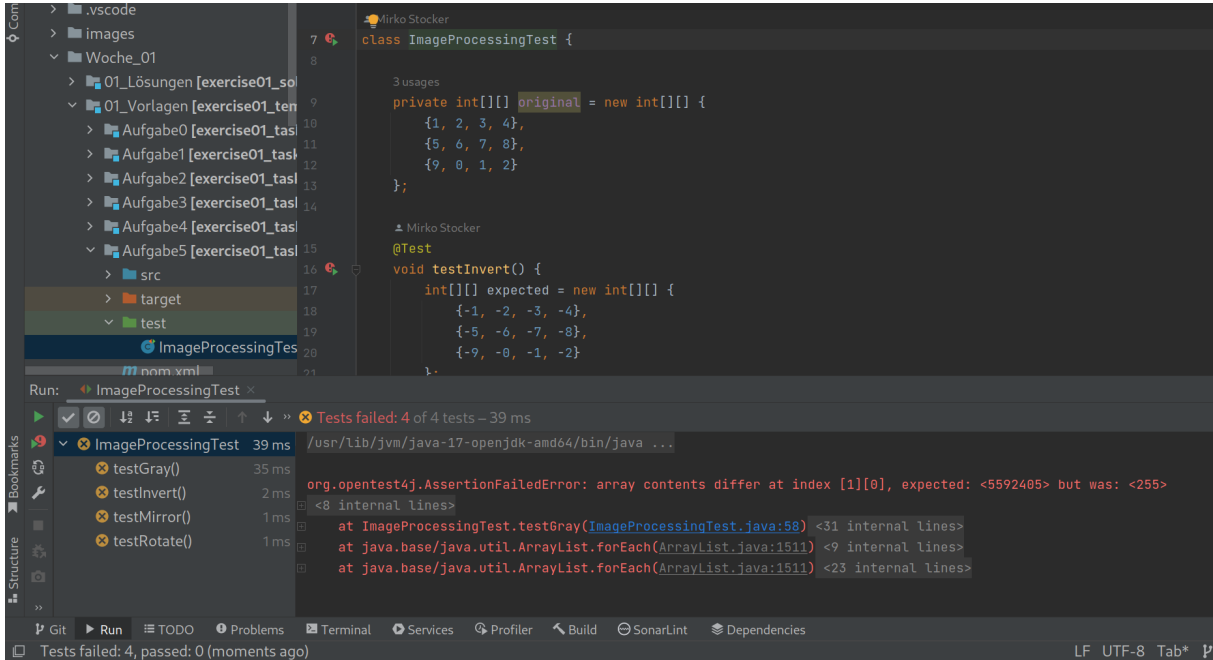
Gehen wir von einem Bild mit 3 x 4 Pixel aus (Achtung: Denkweise/Adressierung $y \times x$). Im Originalbild hat der Pixel bei `pixels[2][0]` den Wert 9.

<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr><tr><td>9</td><td>0</td><td>1</td><td>2</td></tr></table> Originalbild als zweidimensionales Array	1	2	3	4	5	6	7	8	9	0	1	2	<table><tr><td>-1</td><td>-2</td><td>-3</td><td>-4</td></tr><tr><td>-5</td><td>-6</td><td>-7</td><td>-8</td></tr><tr><td>-9</td><td>0</td><td>-1</td><td>-2</td></tr></table> Invertiertes Bild	-1	-2	-3	-4	-5	-6	-7	-8	-9	0	-1	-2	<table><tr><td>9</td><td>5</td><td>1</td></tr><tr><td>0</td><td>6</td><td>2</td></tr><tr><td>1</td><td>7</td><td>3</td></tr><tr><td>2</td><td>8</td><td>4</td></tr></table> Rotiertes Bild in Bezug auf Original	9	5	1	0	6	2	1	7	3	2	8	4	<table><tr><td>4</td><td>3</td><td>2</td><td>1</td></tr><tr><td>8</td><td>7</td><td>6</td><td>5</td></tr><tr><td>2</td><td>1</td><td>0</td><td>9</td></tr></table> Gespiegeltes Bild in Bezug auf Original	4	3	2	1	8	7	6	5	2	1	0	9
1	2	3	4																																																
5	6	7	8																																																
9	0	1	2																																																
-1	-2	-3	-4																																																
-5	-6	-7	-8																																																
-9	0	-1	-2																																																
9	5	1																																																	
0	6	2																																																	
1	7	3																																																	
2	8	4																																																	
4	3	2	1																																																
8	7	6	5																																																
2	1	0	9																																																

Bei dieser Aufgabe erhalten das erste Mal auch Unit Tests, die Sie ausführen können. Sie finden diese im `test`-Verzeichnis und können Sie in IntelliJ über Rechtsklick -> Run 'ImageProcessingTest' (oder mit den beiden grünen Pfeilen im Editor auf der Zeile der Klassendefinition) ausführen.



Dann wird sich automatisch die JUnit Results View öffnen, in welcher Sie die Test-Ergebnisse betrachten können:



```

class ImageProcessingTest {
    3 usages
    private int[][] original = new int[][] {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 0, 1, 2}
    };

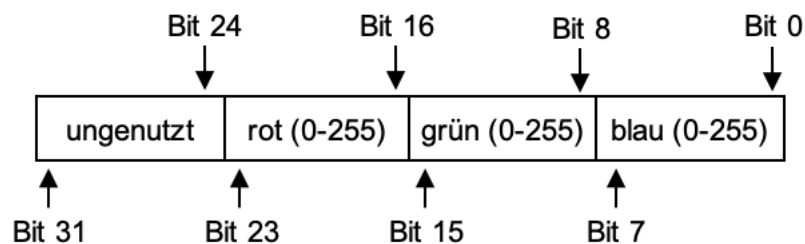
    @Test
    void testInvert() {
        int[][] expected = new int[][] {
            {-1, -2, -3, -4},
            {-5, -6, -7, -8},
            {-9, -0, -1, -2}
        };
    }
}

Run: ImageProcessingTest
Tests failed: 4 of 4 tests – 39 ms
ImageProcessingTest 39 ms
  testGray() 35 ms
  testInvert() 2 ms
  testMirror() 1 ms
  testRotate() 1 ms

org.opentest4j.AssertionFailedError: array contents differ at index [1][0], expected: <5592405> but was: <255>
<8 internal lines>
at ImageProcessingTest.testGray(ImageProcessingTest.java:58) <31 internal lines>
at java.base/java.util.ArrayList.forEach(ArrayList.java:1511) <9 internal lines>
at java.base/java.util.ArrayList.forEach(ArrayList.java:1511) <23 internal lines>
  
```

Sobald die Methoden korrekt implementiert sind, wird der jeweilige Test grün werden.

Optionale Zusatzaufgabe: Als Bonus können Sie einen Graufilter implementieren: Dazu ist es wichtig zu wissen, dass die Pixelwerte im Rot-Grün-Blau-Farbschema (RGB) entsprechend diesem Bit-Layout codiert sind:



Pro Pixel können Sie nun zuerst den Rot-, Grün- und Blauanteil extrahieren und danach den Durchschnitt der drei Anteile bilden. Dieser Durchschnitt dient dann als neuer Rot-, Grün- und Blauanteil des Pixels. Dazu sind Bitoperationen nützlich: eine Übersicht finden Sie unter „Ergänzende Materialien“ in Moodle.

Aufgabe 6: Theoriefragen (Selbststudium)

Welche der Java-Ausdrücke sind gültig? Bestimmen Sie das Ergebnis für die gültigen Ausdrücke. Was ist der Variablenzustand am Schluss? Versuchen Sie die Fragen zuerst ohne Computer bzw. IDE zu lösen.

Es sind folgende Variablen deklariert und initialisiert:

```
int x = 1, y = 2;
```

Ausdrücke:

- a) $2 * x - 3 * y - 1$
- b) $1 - ((x)) + -y$
- c) $1 + -x + -1$
- d) $y * x / y + x \% y$
- e) $x == 0$
- f) $1 < y < 3$
- g) $(x > 0) \&\& (x < 2)$
- h) $x+++x + y-- - --y$